

Llama Point System:

Step 1: Get data from save text file into python:

1. Player Countries
 - a. Get "was player" value
 - b. Find AI signifier for countries that no longer have a player
 - c. Get GP score
2. Player wars
 - a. Length of war
 - b. Was player
 - c. War Object:
 - d. Add_attacker
 - e. add_defender
 - f. Outcome
 - g. Value (participation)
 - h. War start date and end date for every participant

Step 2: Use the data to create llama points

1. Using "was player" and the ai value eliminate non player wars:
 - a. Use the AI value to multiply the $(\text{war par points}[\text{War}][\text{Player}])$ by 0
 - i. ie if (AI value = true) then $(\text{war par points}[\text{War}][\text{Player}]) = 0$;
 - b. OR second method, using a blacklist of linked Player-Nation-Date-Join/Leave combos we eliminate all contributions to the points from them after they left or joined
 - i. ie if if country is = France AND player = Llama AND war is after 1470 then continue, otherwise stop.
 - ii. This would be the example if i had hotjoined in 1470
2. War participation will be used as follows for each war: $(\text{war par points}[\text{War}][\text{Player}])$
 - a. Order the "participation" value from each war from greatest to least.
 - b. Use that order to order the tags that correspond to those participation values.
 - i. If [Player] attacking side and outcome = 2 then Win Points
 - ii. If [Player] attacking side and outcome = 3 then Lose Points
 - iii. If [Player] attacking side and outcome = 1 then No Points
 - iv. If [Player] defending side and outcome = 2 then Lose Points
 - v. If [Player] defending side and outcome = 3 then Win Points
 - vi. If [Player] defending side and outcome = 1 then No Points
 1. Win Points: #n war participation on the winning side gets $(\text{war par points}[\text{War}][\text{Player}]) = ((N_{\text{win}}[\text{War}] - (n - 1)) * (\text{victory points}))$
etc...until $((N_{\text{win}}[\text{War}] - (n - 1)) = 1)$
 2. Lose Points: All war participants on the losing side gets $(\text{war par points}[\text{War}][\text{Player}]) = (\text{sum of losing war participants}[\text{War}] / (N_{\text{lose}}[\text{War}])) * (\text{victory points}))$
3. Scale values will be used: $(\text{war scale factor}[\text{War}])$

- a. Where $(\text{war scale factor}[\text{War}]) = \text{Log}[(\text{sum of total participation for that war (both sides combined)})]$
4. Now for the value that each player gets for the campaign so far: $(\text{Total War Value}[\text{Player}])$
 - a. $(\text{Total War Value}[\text{Player}]) = \text{Sum over wars } (\text{war scale factor}[\text{War}]) * (\text{war par points}[\text{War}][\text{Player}])$ for each player
 - b. Each player will be assigned this value ie. Llama -> $(\text{Total War Value}[\text{Llama}])$
5. Now finally for the $(\text{Llama Points}[\text{Player}])$
 - a. $\text{Log}[\text{GP Score}[\text{Player}]] + (\text{Total War Value}[\text{Player}])$
6. Now display the $(\text{Llama Points}[\text{Player}])$ in a readable format
 - a. ie Llama (20), Ruben (30), DiePie (30)
 - b. Maybe a bar graph if we have time?

Technical coding plans:

Player, country, score stored as object

Ex: `penn.getScore()`; or `penn.score`;

Llama score follows from country to country

Co-ops: Two war participants with equal participation

Wars are objects, shared between player objects

Each Player has a list of Wars. Each War has a list of Players which are assigned participation score and outcome value (maybe have a separate PlayerData object, which is assigned to each player in a map?). Only players with participation of ≥ 100 are added to the war, AI is ignored.

If the save finds that a player has formed/released as/whatever a nation, it should automatically add that to their Player so that the person running the program doesn't have to manually enter it.

Loadable save file(s) with when players were in-game?

NOTE: All countries should be saved as their IDs, in case there's a name switch (eg, Kingdom of God is formed). When Skander's Llama score is correctly calculated, we know we've done it.

GAME PLAN:

Phase 1:

1. Get data from the save .txt file into a program ✓

2. IDEA:

- a. Maybe make a method that is a generic scanner, we give it two arguments (startscanning, endscanning) and it outputs whatever was between those two points. Then make it repeat every time it sees the starting text and end when it sees the ending text putting all of the values into an array. This could be a way to

get data out of the sheet efficiently. Could also be reusable multiple times for every string of data we need to get out of the sheet instead of having to make a fresh method each time.

3. Get every date that every participant joined and left every war
 - a. So we would need to put under player (war,date1,date2)
 - b. And for every war (player, date1,date2)
4. Use this data to create a duration of time that every war occurs for each player
 - a. I was using just a number since 1444 but it might be easier to do with LocalDate idk that's up to you I don't know how its methods work
5. Create a system where you input the duration of playtime on each tag that corresponds to each player over the course of the campaign (i think you said the max you could do was two tags which is fine)
6. Use this system to cross check if the player was active on the nation during the war
 - a. If they are then let the war count for that player and if not then remove them

Phase 2:

1. Now we assign an outcome to each war
2. Assign attacking or defending to each player for each war
3. Assign a participation (value) for each player for each war
4. Use the score equations to create a scoring system outlined above (FINALLY SOMETHING I CAN DO)
5. Finally we assign the scores that were created to each player
6. Use graphics? to display the scores next to the player names

OVERALL:

PLAYER(list of wars participated, list of attacking or defending, list of join dates, list of end dates, list of participation for each war, GP score, final score)

WAR(list of players in war, list of attacking or defending for each player, list of time range of each player, list of participation for each player, war outcome)

You can organize this better by putting the war object within the player object?

I am using matrix notation for the objects cuz idk what else to do but these are objects with multiple lists of values in them as well as single values. I want to do this chronologically so that all the Y axis line up, for example:

Wars:	A/D:	Start:	End:	Partici:
Wars.getLlama() =(conquest war 1	attacking	1444.11.11	1444.11.12	10000
Conquest war 2	defending	1444.12.13	1444.12.14	13200
Conequest war 3	defending	1445.1.13	1445.2.13	132300

Outcome:

1
2
3)

	GP Score:	Llama Score
Players.getLlama() = (Wars.getLlama()	10232	100)